AC02-3Tri-IoT

ARMÁRIO INTELIGENTE COM RFID E MQTT

Ana Elisa - Raphael Ramalho

PROBLEMA E PROPOSTA

Em **escolas**, **academias** e **empresas**, armários compartilhados costumam usar chaves ou cadeados, que podem ser perdidos, copiados ou emprestados sem nenhum registro. Também não existe um histórico de quem abriu o armário e quando, o que dificulta a gestão quando um cartão é perdido ou quando alguém tenta abrir um armário que não é seu.

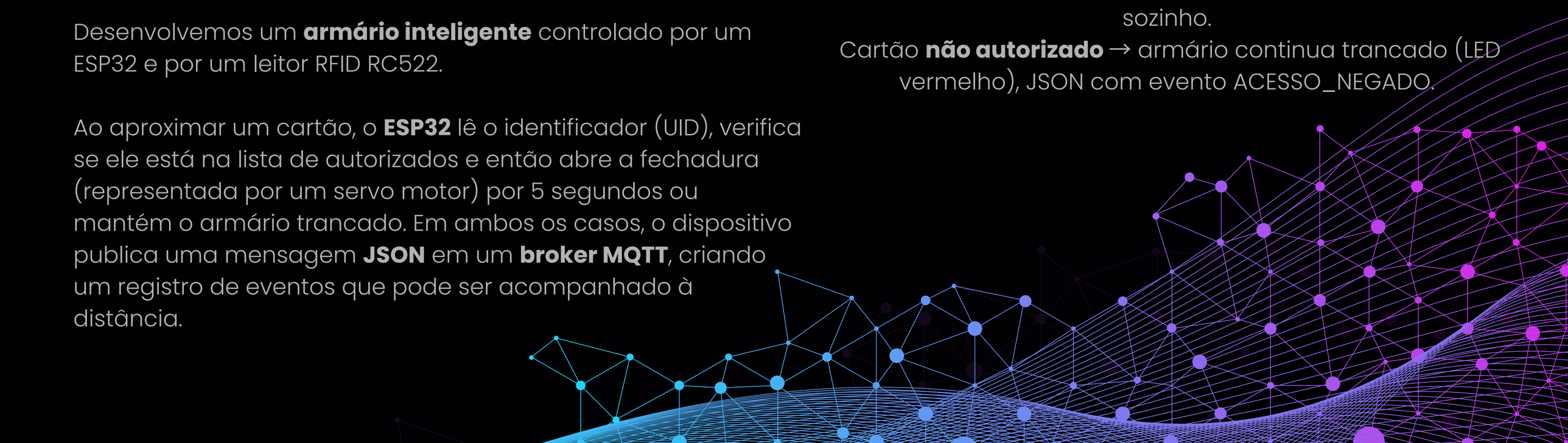
Desenvolvemos um **armário inteligente** controlado por um ESP32 e por um leitor RFID RC522.

Ao aproximar um cartão, o **ESP32** lê o identificador (UID), verifica se ele está na lista de autorizados e então abre a fechadura (representada por um servo motor) por 5 segundos ou mantém o armário trancado. Em ambos os casos, o dispositivo publica uma mensagem **JSON** em um **broker MQTT**, criando um registro de eventos que pode ser acompanhado à distância.

Comportamento esperado:

Cartão **autorizado** → servo gira para 90º, LED verde acende, JSON com evento ACESSO_LIBERADO, após 5s o armário tranca sozinho.

Cartão **não autorizado** → armário continua trancado (LED vermelho), JSON com evento ACESSO_NEGADO.



RFID E IDENTIFICAÇÃO

RFID (Radio Frequency Identification) é uma tecnologia que identifica objetos por ondas de rádio, sem contato físico.

O sistema tem duas partes: o leitor e a etiqueta (cartão, chaveiro ou adesivo).

Segundo a folha de dados da NXP (2016), o MFRC522 é um leitor que opera em 13,56 MHz e se comunica com cartões do padrão ISO/IEC 14443 A, como os da família MIFARE. A comunicação com o microcontrolador é feita por SPI, que é o barramento usado no projeto.

Os cartões usados são **passivos**: não possuem bateria. Quando o cartão se aproxima, o campo eletromagnético gerado pelo leitor induz energia na antena do cartão, que alimenta o chip e permite que ele responda ao leitor.

Cartão no Wokwi	UID
Blue Card	01:02:03:04
Green Card	11:22:33:44
Yellow Card	55:66:77:88
Red Card	AA:BB:CC:DD
NFC Tag	04:11:22:33:44:55:66 (7 bytes)
Key Fob	C0:FF:EE:99

LIMITAÇÕES E RISCOS

Clonagem do UID: o UID é transmitido sem criptografia. Um sistema que decide o acesso somente pelo UID, como este protótipo didático, pode ser enganado por um cartão clonado. Sistemas reais usam autenticação com chaves criptográficas.

Broker público: o broker broker.hivemq.com é aberto, e qualquer pessoa que souber o tópico pode ler as mensagens. A documentação do Wokwi (s.d.) também avisa que o gateway público de rede é monitorado e não deve ser usado para dados privados. Por isso o projeto transmite apenas dados fictícios e nenhuma senha real.

Alcance curto: o leitor só detecta cartões a poucos centímetros, o que é bom para segurança, mas exige que o cartão seja posicionado corretamente.

CAMINHO DOS DADOS

Entrada	ESP32	Wi-Fi	Broker MQTT	Visualização
Cartão RFID aproximado do leitor RC522 (SPI)	Lê UID e SAK, decide acesso, aciona servo e LEDs, monta o JSON	Rede virtual Wokwi-GUEST	broker.hivemq.com om porta 1883, tópico armario/ramalho/A01	HiveMQ Web Client assinando o tópico

Segundo a **HiveMQ**, o **MQTT** funciona no modelo **publicar/assinar**: o dispositivo publica mensagens em um tópico, o broker recebe e entrega a todos os clientes que assinaram esse tópico.

No projeto, o ESP32 é o publicador e o **HiveMQ Web Client** é o assinante. Usamos um tópico próprio (**armario/ramalho/A01**) para não misturar mensagens com as de outros grupos.

EXEMPLOS DE USO REAL

RFID passivo em 13,56 MHz é usado em crachás de empresas e escolas, cartões de transporte público e armários de academias: o leitor identifica o cartão e o sistema decide o acesso e registra o evento.

LIGAÇÕES DO CIRCUITO

Componente	Pino do componente	Pino do ESP32
Leitor RFID RC522	SDA / SCK / MOSI / MISO / RST	5 / 18 / 23 / 19 / 21
Leitor RFID RC522	3.3V / GND	3V3 / GND
Servo (fechadura)	PWM / V+ / GND	13 / 5V / GND
LED verde (aberto)	Anodo via resistor 220 Ω	26
LED vermelho (trancado)	Anodo via resistor 220 Ω	27

EXEMPLOS DE JSON PUBLICADOS

Cartão autorizado (Blue Card):

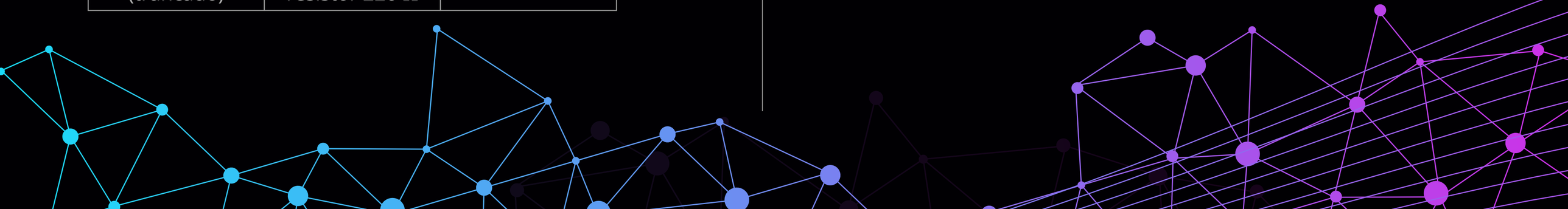
{"armario":"A01","evento":"ACESSO_LIBERADO","uid":"01:02:03:04",
"tipo_cartao":"MIFARE 1KB","leituras":1}

Cartão não autorizado (Red Card):

{"armario":"A01","evento":"ACESSO_NEGADO","uid":"AA:BB:CC:DD",
"tipo_cartao":"MIFARE 1KB","leituras":2}

Cartão de outro tipo (Key Fob):

{"armario":"A01","evento":"ACESSO_NEGADO","uid":"C0:FF:EE:99",
"tipo_cartao":"MIFARE Mini, 320 bytes","leituras":5}



A MELHORIA

o exemplo da catraca visto em aula, apertava um botão e liberava um Servo, apenas uma mensagem simples de acesso.

A melhoria do grupo muda o tratamento dos dados e a publicação MQTT: o **ESP32** passa a identificar o tipo de **cartão** pelo byte SAK, monta o UID com qualquer número de bytes, conta as leituras e publica um **JSON** com quatro informações:
identificador (uid)
tipo do cartão (tipo_cartao)
tipo de evento (evento)
contador de leituras (leituras)

Com isso, quem acompanha o tópico sabe não só quem tentou abrir, mas que tipo de credencial foi usada e quantas leituras já ocorreram.

RESULTADOS DOS TESTES

Teste	Entrada	Resultado esperado	Resultado	Evidência
1. Primeira situação	Blue Card (01:02:03:04), Tap	JSON com evento ACESSO_LIBERADO no tópico armario/ramalho/A01; servo abre 5 s	OK. ACESSO_LIBERADO recebido no HiveMQ; armário abriu e trancou após 5 s.	Fig. 2 e 6
2. Segunda situação	Red Card (AA:BB:CC:DD), Tap	JSON com evento ACESSO_NEGADO; armário continua trancado	OK. ACESSO_NEGADO recebido; armário continuou trancado.	Fig. 3 e 6
3. Limite ou falha	Blue Card mantido no leitor (chave Hold ligada) por cerca de 15 s	Apenas UMA mensagem publicada, pois o cartão é colocado em espera (PICC_HaltA) e não é relido enquanto fica parado	OK. Só uma mensagem (leituras 3, às 09:31:55). Nenhuma mensagem repetida durante os 15 s com o cartão parado.	Fig. 4 e 6
4. Melhoria do grupo	NFC Tag e depois Key Fob, Tap	tipo_cartao muda (Ultralight / Mini), UID de 7 bytes da NFC Tag aparece completo e leituras aumenta	Parcial. Tipo do cartão identificado (Ultralight e Mini) e contador subiu, mas o UID da NFC Tag veio com 4 bytes em vez de 7.	Fig. 5 e 6

ANÁLISE DA FALHA DO TESTE 4

O tipo do cartão foi identificado corretamente, o que comprova a melhoria, mas o **UID de 7 bytes da NFC Tag** chegou cortado em **4 bytes**. Em cartões de **7 bytes**, o **UID** é lido em dois níveis de seleção. Nossa hipótese é que apenas o primeiro nível foi devolvido nessa leitura. O código que monta o **UID** percorre todos os bytes informados por **rfid.uid.size**, então o corte vem da leitura, não da montagem do texto. Como próximo passo, o grupo pretende imprimir **rfid.uid.size** no **Serial Monitor** para confirmar a causa.

As capturas de tela de cada teste estão no Apêndice A, ao final do relatório.
Dificuldade encontrada durante o desenvolvimento: na primeira versão, o **ESP32** reiniciava sem parar com a mensagem "**assert failed**", porque o código tentava conectar ao broker antes de conectar ao **Wi-Fi**. Após incluir a conexão **Wi-Fi no setup()**, a placa passou a funcionar. Em seguida, as mensagens não chegavam ao **HiveMQ** porque o trecho de conexão com o **broker** não estava no início do **loop()**, depois de movê-lo para lá, a conexão passou a ser feita corretamente.

CONCLUSÃO

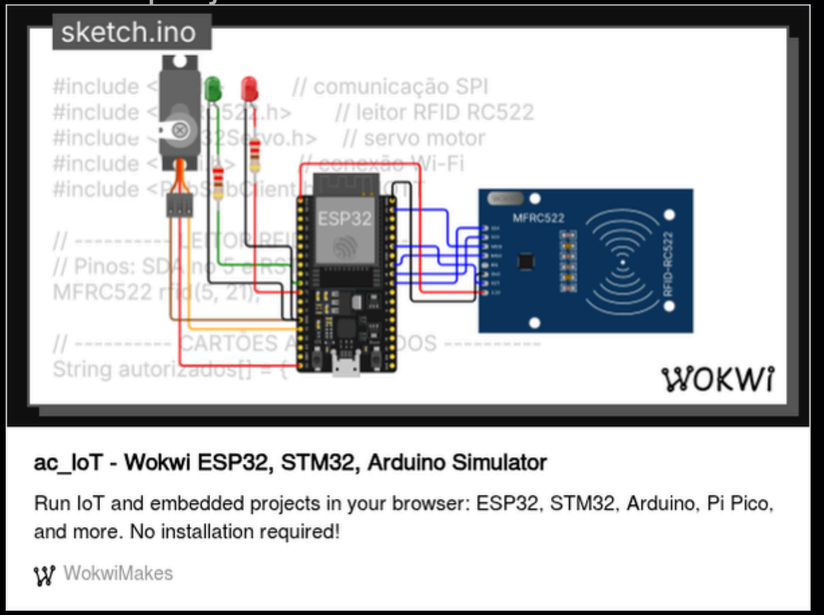
O que funcionou: a leitura dos cartões no Wokwi, a decisão de acesso, o acionamento do servo e dos LEDs e a publicação dos JSONs no broker, comprovada pelo HiveMQ Web Client. A melhoria permitiu diferenciar os tipos de cartão e acompanhar o número de leituras.

O que pode melhorar: usar autenticação por chave no cartão em vez de confiar apenas no UID; usar um broker privado com usuário, senha e conexão criptografada (TLS); separar os eventos em tópicos diferentes; evitar o delay() de 5 s, que impede leituras enquanto o armário está aberto; e corrigir a leitura do UID de 7 bytes observada no teste 4.

USO DE CÓDIGOS E FERRAMENTAS DE APOIO

Exemplos das bibliotecas MFRC522, ESP32Servo e PubSubClient. Foram alterados o cenário (armário), a fechadura com servo, a lista de autorizados, a montagem do JSON e a identificação do tipo de cartão. O grupo utilizou um assistente de IA como apoio para explicar os conceitos passo a passo, encontrar erros no código, executar os testes. As respostas foram verificadas testando cada etapa no Wokwi, conferindo as mensagens no HiveMQ e conferindo as informações nas fontes listadas abaixo.

Link do projeto:



REFERÊNCIAS

- Wokwi – MFRC522 RFID Reader
- Wokwi – ESP32 WiFi Networking
- HiveMQ – MQTT Essentials Part 4
- NXP – MFRC522 datasheet
- NXP – AN10833 MIFARE type identification
- Biblioteca MFRC522 (miguelbalboa/rfid)

FIGURA 1 – SERIAL MONITOR APÓS A CONEXÃO COM O BROKER

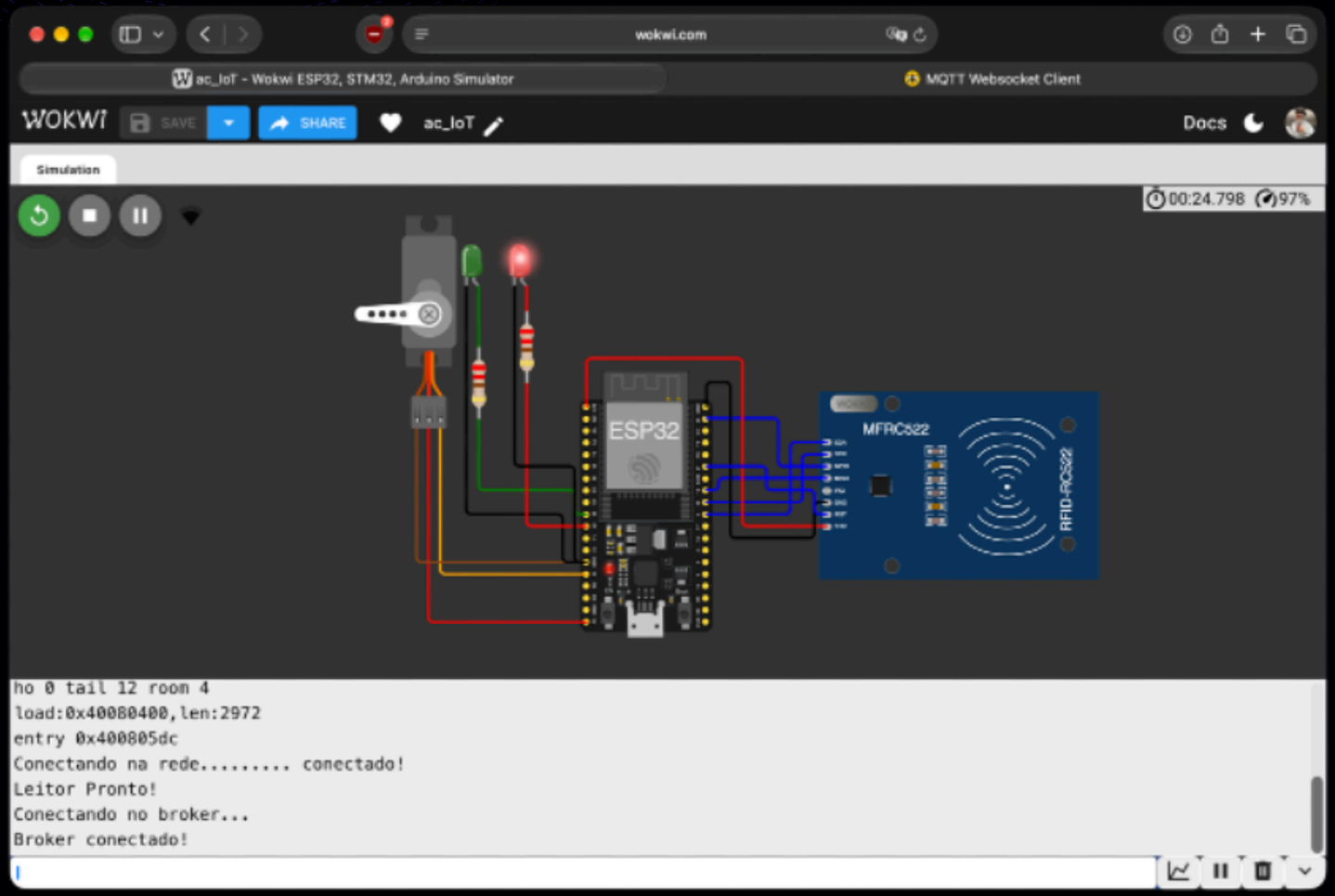


FIGURA 2 – TESTE 1: BLUE CARD, ACESSO_LIBERADO

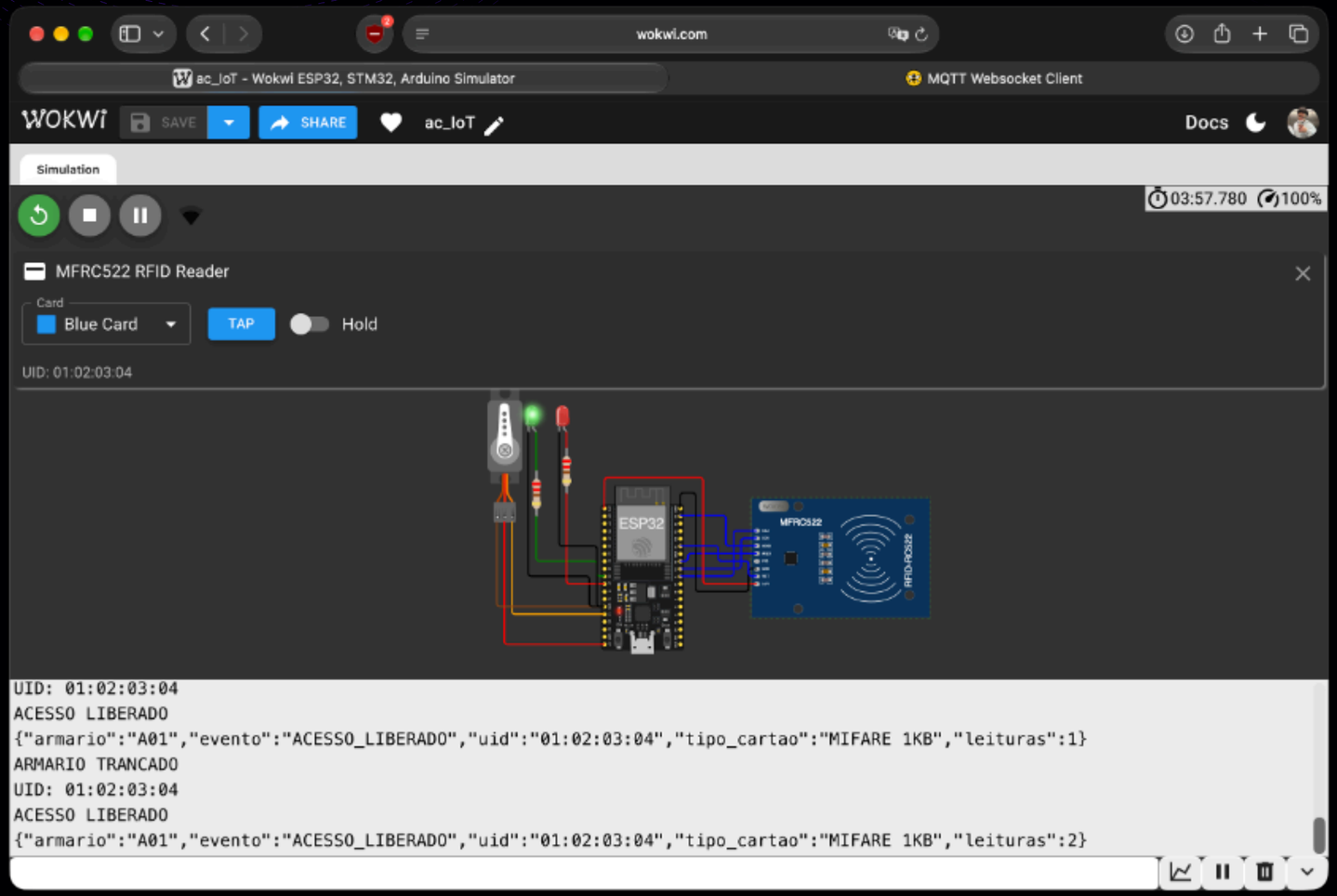


FIGURA 3 – TESTE 2: RED CARD, ACESSO_NEGADO

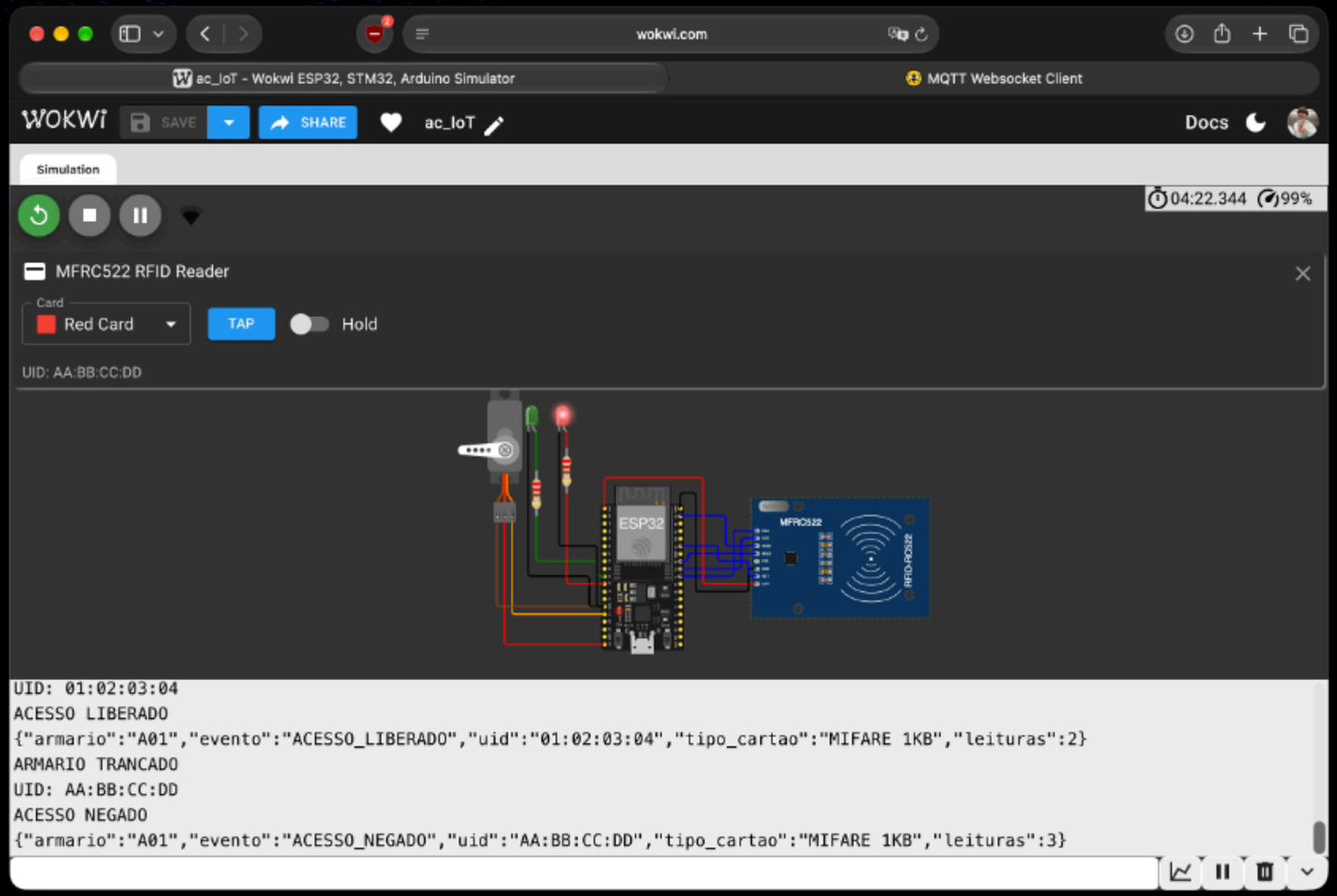


FIGURA 4 – TESTE 3: BLUE CARD MANTIDO COM HOLD

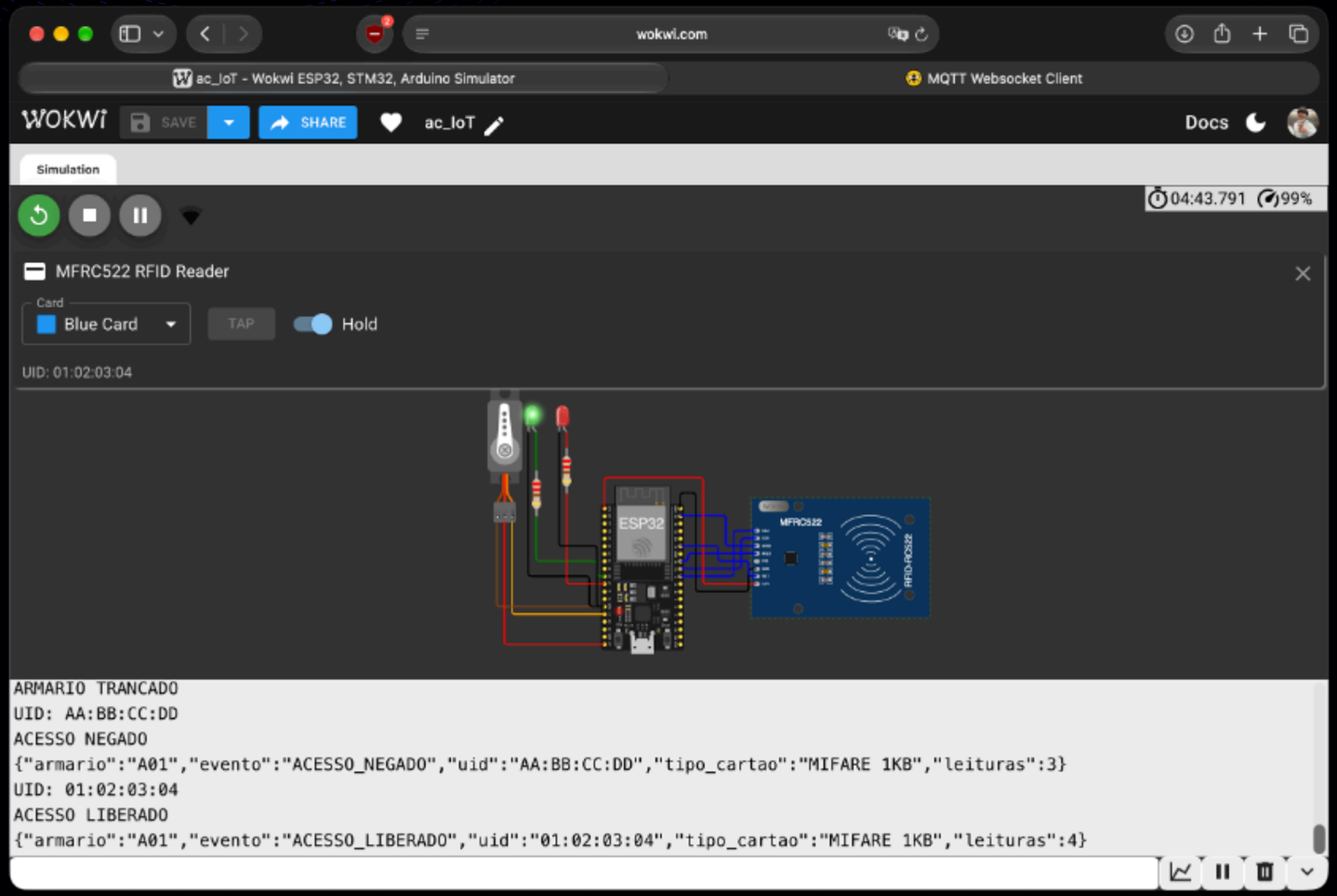


FIGURA 5 – TESTE 4: KEY FOB

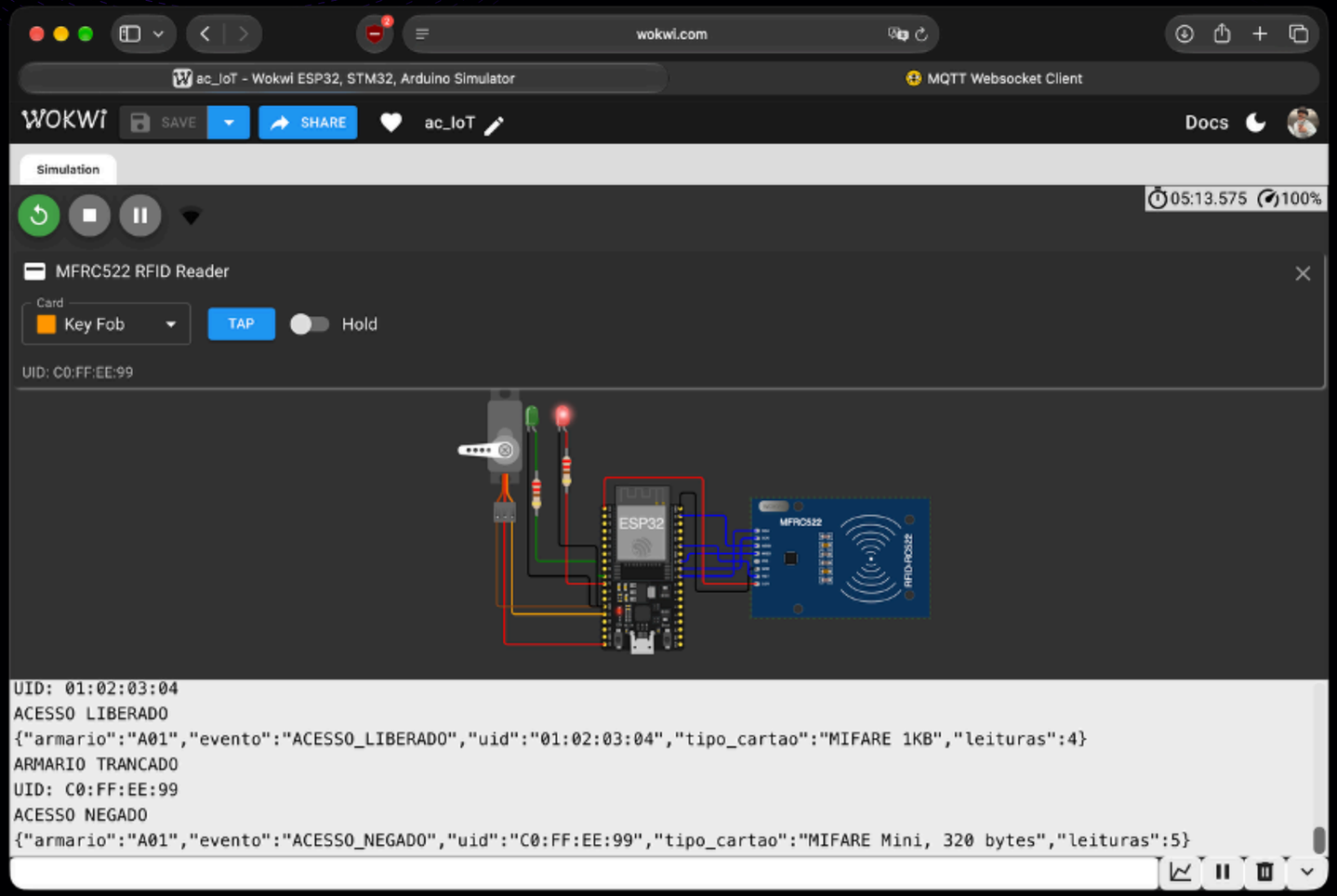
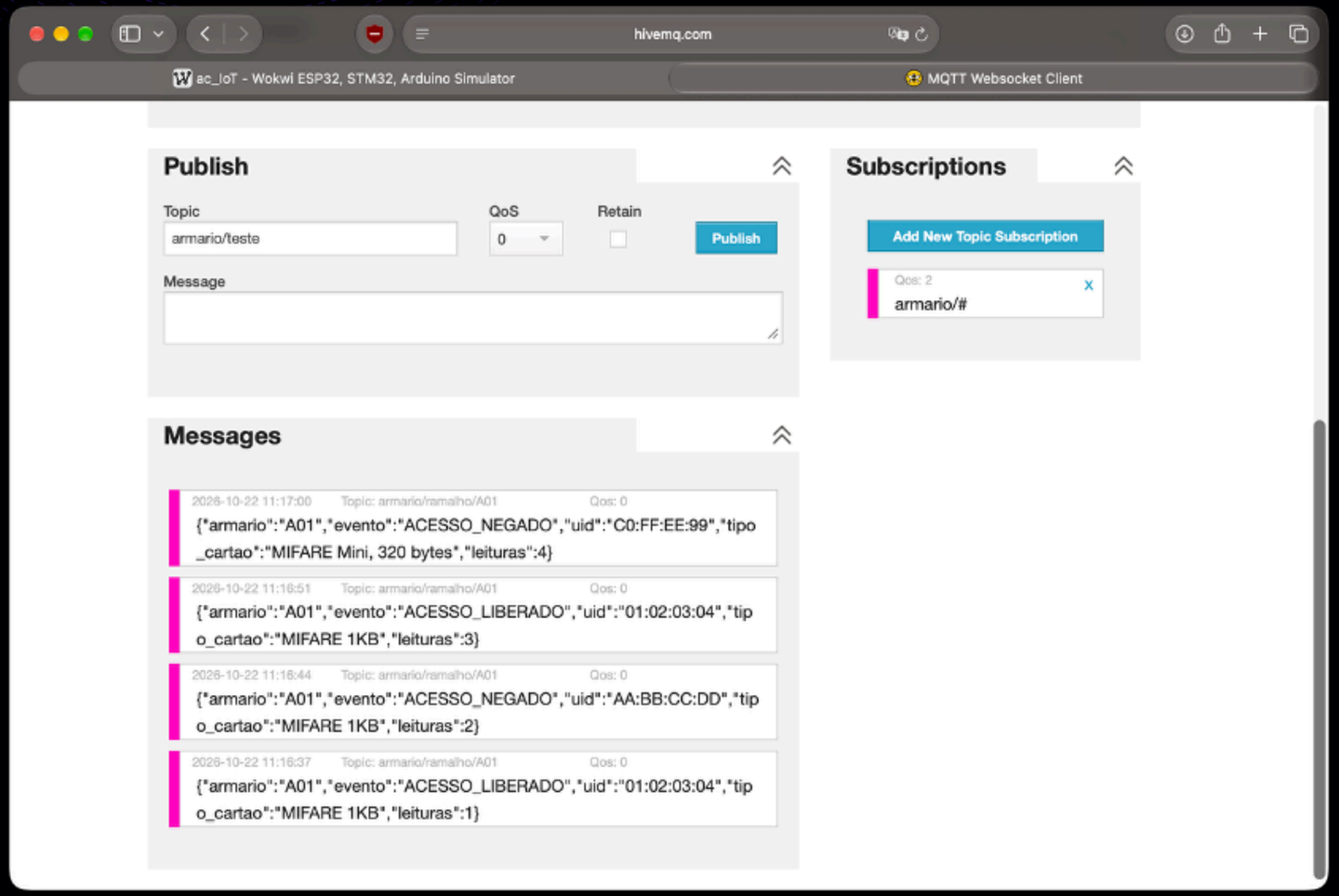


FIGURA 6 – MENSAGENS RECEBIDAS NO HIVEMQ WEB CLIENT



RECADO IMPORTANTE !!

Yan, pfv corrija com carinho e lembre-se que: eu (Ramalho) e a Ana, **NUNCA ARRASTAMOS NA SUA AULA,** e tbm é a ultima AC de IoT do ano, faz a boa chefe =)